

chordx

A package to write song lyrics with chord diagrams in Typst.

v0.7.0

May 3, 2026

<https://github.com/ljgago/typst-chords>

Leonardo Javier Gago

Contents

1. Reference	3
1.1. Chart Chords	3
1.1.1. chart-chord	3
1.2. Piano Chords	6
1.2.1. piano-chord	6
1.3. Single Chords	8
1.3.1. single-chord	8
1.4. Chorded Stanza	10
1.4.1. chorded-stanza	10
1.4.2. Example	10
1.4.3. Custom chord typesetting	11
1.4.4. Special scenarios	12

1. Reference

1.1. Chart Chords

- [chart-chord\(\)](#)

1.1.1. chart-chord

Generates a chart chord for stringed instruments.

Parameters

```
chart-chord(
  ..text-params: auto,
  tabs: str,
  fingers: str,
  capos: str,
  fret: none int,
  frets-amount: int,
  design: str,
  position: str,
  background: color,
  name: str content
) -> content
```

..text-params **auto**

Embeds the native **text** parameters from the standard library of **typst**. **Optional**.

tabs **str**

Shows the tabs on the chart. **Optional**.

- **x**: mute note.
- **o**: air note.
- **n**: without note.
- **number**: note position on the fret.

The string length of tabs defines the number of strings on the instrument.

Example:

- "x32o1o" - (6 strings - C Guitar chord).
- "ooo3" - (4 strings - C Ukulele chord).

Default: ""

fingers **str**

Shows the finger numbers. **Optional**.

- **n, x, o**: without finger,
- **number**: one finger

Example: "n32n1n" - (Fingers for guitar chord: C)

Default: ""

capos `str`

Adds one or many capos on the chart. **Optional.**

- 1st digit – **fret**: fret position.
- 2nd digit – **start**: lowest starting string.
- 3rd digit – **end**: highest ending string.

Example: "115" $\equiv$ "1,1,5" $\Rightarrow$ "fret,start,end"

With "|" you can add capos:

Example: "115|312" $\equiv$ "1,1,5|3,1,2" $\Rightarrow$ "fret,start,end|fret,start,end"

Default: ""

fret `none` or `int`

Shows the fret number that indicates the starting position of the fretboard. **Optional.**

Default: `none`

frets-amount `int`

Sets the frets amount (the grid rows). **Optional.**

Default: `5`

design `str`

Sets the chart design. **Optional.**

- `"sharp"`: chart with sharp corners.
- `"round"`: chart with round corners.

Default: `"sharp"`

position `str`

Sets the chord chart position. **Optional.**

- `"top"`: chord chart in top position.
- `"bottom"`: chord chart in bottom position.

Default: `"top"`

background `color`

Sets the background color of the chord name. **Optional.**

Default: `rgb(0, 0, 0, 0)`

name `str` or `content`

Shows the chord name. **Required.**

Example:

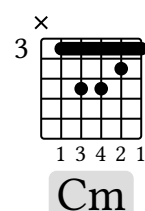
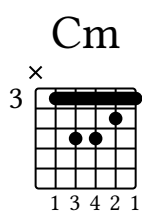
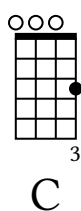
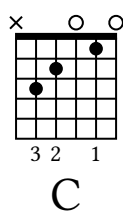
```
#import "@preview/chordx:0.7.0": chart-chord

#let chart-chord-sharp = chart-chord.with(size: 18pt)
#let chart-chord-round = chart-chord.with(size: 18pt, design: "round")

// Design "sharp"
#chart-chord-sharp(tabs: "x32o1o", fingers: "n32n1n")[C]
#chart-chord-sharp(tabs: "ooo3", fingers: "ooo3")[C]

// Design "round" with position "bottom"
#chart-chord-round(tabs: "xn332n", fingers: "o13421", fret: 3, capos: "115", position: "bottom")[Cm]
#chart-chord-round(tabs: "onnn", fingers: "n111", capos: "313", position: "bottom")[Cm]

// Design "round" with background color in chord name
#chart-chord-round(tabs: "xn332n", fingers: "o13421", fret: 3, capos: "115", background: silver)[Cm]
#chart-chord-round(tabs: "onnn", fingers: "n111", capos: "313", background: silver)[Cm]
```



1.2. Piano Chords

- [piano-chord\(\)](#)

1.2.1. piano-chord

Generates a piano chord.

Parameters

```
piano-chord(
  ..text-params: auto,
  keys: str,
  layout: str,
  fill-key: color,
  design: str,
  position: str,
  background: color,
  name: str content
) -> content
```

..text-params `auto`

Embeds the native **text** parameters from the standard library of **typst**. **Optional**.

keys `str`

Keys chord notes from **C1** to **E3** (Depends on your layout). **Optional**.

Example: "**C1**, **E1b**, **G1**" (Cm chord)

Default: ""

layout `str`

Sets the layout and size of the piano, "**C**", "**2C**", "**F**", "**2F**". **Optional**.

- "**C**": the piano layout starts from key **C1** to **E2** (17 keys).
- "**2C**": the piano layout starts from key **C1** to **B2** (24 keys, two octaves).
- "**F**": the piano layout starts from key **F1** to **B2** (19 keys).
- "**2F**": the piano layout stars from key **F1** to **E3** (24 keys, two octaves).

Default: "**C**"

fill-key `color`

Sets the fill color of the pressed key. **Optional**.

Default: gray

design `str`Sets the piano design. **Optional.**

- `"sharp"`: piano with sharp corners.
- `"round"`: piano with round corners.

Default: `"sharp"`**position** `str`Sets the chord chart position. **Optional.**

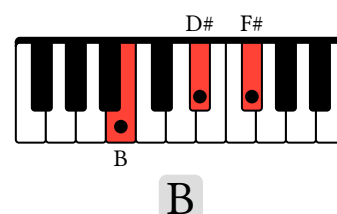
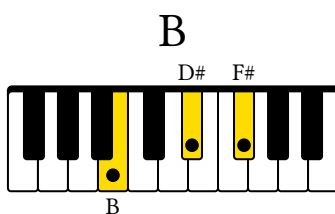
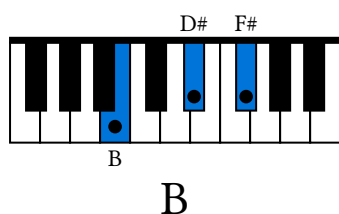
- `"top"`: chord chart in top position.
- `"bottom"`: chord chart in bottom position.

Default: `"top"`**background** `color`Sets the background color of the chord name. **Optional.**Default: `rgb(0, 0, 0, 0)`**name** `str` or `content`Shows the chord name. **Required.****Example:**

```
#import "@preview/chordx:0.7.0": piano-chord

#let piano-chord-sharp = piano-chord.with(layout: "F", size: 18pt)
#let piano-chord-round = piano-chord.with(layout: "F", size: 18pt, design: "round")

#piano-chord-sharp(keys: "B1, D2#, F2#", fill-key: blue)[B]
#piano-chord-round(keys: "B1, D2#, F2#", fill-key: yellow, position: "top")[B]
#piano-chord-round(keys: "B1, D2#, F2#", fill-key: red, background: silver)[B]
```



1.3. Single Chords

- [single-chord\(\)](#)

1.3.1. single-chord

The single chord a chord without diagram used to show the chord name over a word.

Parameters

```
single-chord(
  ..text-params: auto,
  inner-gap: length,
  outer-gap: length,
  background: color,
  body: content,
  name: content,
  position: content
) -> content
```

..text-params `auto`

Embeds the native **text** parameters from the standard library of **typst**. **Optional**.

inner-gap `length`

Sets the inner gap between the bottom word and the chord name. **Optional**.

Default: `0.2em`

outer-gap `length`

Sets the outer gap between the upper paragraph and the chord name. **Optional**.

Default: `0.6em`

background `color`

Sets the background color of the chord. **Optional**.

Default: `rgb(0, 0, 0, 0)`

body `content`

Is the word or words where the chord goes. **Required**.

name `content`

Displays the chord name over the selected words in the body. **Required**.

1.4. Chorded Stanza

- [chorded-stanza\(\)](#)

1.4.1. chorded-stanza

A helper method, to simplify entering entire stanzas, without having to type a complete single-chord command for each chord.

Parameters

```
chorded-stanza(
    ..single-chord-params: auto,
    preserve-linebreaks: bool,
    custom-
typesets: dictionary (Key: `str` with chord name (as in the input); Value: `content`),
    content: str
) -> content
```

..single-chord-params `auto`

Embeds all the parameters of the single-chord function, including the native **text** parameters from the standard library of **typst**. **Optional**.

preserve-linebreaks `bool`

If true, linebreaks of the input will be replicated in the output. If false, the output will be without explicit linebreaks. **Optional**.

Default: `true`

custom-typesets

dictionary (Key: `str` with chord name (as in the input); Value: `content`)

Dictionary with custom typesets for chords.

Default: `(:)`

content `str`

The stanza. Lines of chords and lines of text alternating. **Required**.

1.4.2. Example

Consider above example on printing “Jingle Bells” with chords (Section 1.3.1.2). `chorded-stanza` provides a more comfortable way of entering the song:

```
#import "@preview/chordx:0.7.0": chorded-stanza

#let stanza = chorded-stanza.with(
  font: "PT Sans",
  size: 10pt,
  weight: "semibold",
  background: silver,
  preserve-linebreaks: true
)

#stanza("
  G              C      G
Jingle bells, jingle bells, jingle all the way!
  C              G
Oh what fun it is to ride
    A7          D7
In a one-horse open sleigh, hey!
")
```

G
C
G
 Jingle bells, jingle bells, jingle all the way!
C
G
 Oh what fun it is to ride
A7
D7
 In a one-horse open sleigh, hey!

In any text-editor with a mono spaced font, you just enter the text with the cords in the line above. The chorded-stanza function will automatically convert it into corresponding calls to the single-chord function.

Note: The input has to be a pure String (hence the parantheses and quotation marks instead of square brackets) and consist of an even number of lines, where the odd numbered lines contain the chords and the even numbered lines the lyrics.

Note: The input string may be any valid UTF-8 string. Multi-byte characters are counted correctly.

Note: Automatically centering a chord on a word (as done in single-chord by leaving the position empty) is not possible this way, because the first character of the chord name is always exactly above one character of a lyric word.

1.4.3. Custom chord typesetting

If you want to print chord names with special typesets, e.g. ‘C_b5’, that can’t be input directly, you can provide a mapping between the input string and the desired out output string.

```
#import "@preview/chordx:0.7.0": chord-stanza

#let stanza = chorded-stanza.with(
  font: "PT Sans",
  size: 10pt,
  weight: "semibold",
  background: silver,
  preserve-linebreaks: false,
  custom-typesets: (
    "D9": [D#super[9]],
    "C+": [C#super[+]],
    "G#": [G#symbol.sharp]
  )
)

#stanza("
D9   C+       G#
Lorem ipsum dolor sit amet,
F9       E+       A#
consetetur sadipscing elitr...
")
```

D⁹ **C⁺** **G[#]** **F⁹** **E⁺** **A[#]**
 Lorem ipsum dolor sit amet, consetetur sadipscing elitr...

Note: The difference in the rendering between the first and the second sets of chords is the usage of custom typeset mappings.

Note: In this example, `preserve-linebreaks` has been disabled, so that the two input lines are rendered as one output line.

1.4.4. Special scenarios

The `chorded-stanza` function can handle a few special scenarios. While this section is mostly about the internal working of the `chorded-stanza` function, it might be of relevance when you're applying more complex design features (e.g. automatically placing a frame around each individual word).

Merging words

In a situation, where the name of a chord, is longer than the word itself (such that it hangs over the next chord), the two words will be merged into one. For example.:

```
#stanza("
Esus4
Oh what ...
")
```

will be merged into a single word that is handed to `single-chord`:

```
#chord[Oh what][Esus4][1] ...
```

resulting in:

Esus4
 Oh what ...

This differs from the unmerged version:

```
#chord[Oh][Esus4][1] what ...
```

which would result in an unnecessary long whitespace:

Esus4

Oh what ...

You can avoid the merging of words, by increasing the spaces on the input:

```
#stanza("
Esus4
Oh    what ...
")
```

Splitting words

If you have a word that is so long that it needs multiple chords above it, it will be split automatically into multiple sub-words, which are passed to the `single-chord` function. Let's use the German word for DNA as an example:

```
#stanza("
A    D    G    C
Desoxyribonukleinsäure
")
```

will be converted to:

```
#chord[Desoxy][A][1]#chord[ribo][D][1]#chord[nuklein][G][1]#chord[säure][C][1]
```

and rendered like:

A D G C⁹
Desoxyribonukleinsäure